

Metropolis-Hastings Notes

M08: Lecture 5 (Pages 14-18)

Callum Arnold

22 July, 2021

Contents

Ergodic Theorem	1
Markov chain Monte Carlo	1
Metropolis -Hastings	2

Ergodic Theorem

- If you have an irreducible Markov chain $(\{X_n\})$ and positive recurrent (means that the random walks don't go off to infinity) with stationary distribution
 - Let $f : E \rightarrow \mathbb{R}$ be an arbitrary function that maps Markov chain states to real numbers
 - * Satisfying $\sum_{i \in E} |f(i)|\pi_i < \infty$
 - $\lim_{N \rightarrow \infty} \underbrace{\frac{1}{N} \sum_{k=1}^N f(X_k)}_{\text{time average}} = \sum_{i \in E} f(i)\pi_i = \underbrace{E_{\pi}[f(X)]}_{\text{space average}}$
 - * Average of the states plugged into the function is equal to the expectation (average) of the stationary distribution with respect to the function
 - Average across time steps = average across states of the system
- For example, if:

$$f(X_n) = \begin{cases} 1 & \text{if } X_n = 3 \\ 0 & \text{if } X_n \neq 3 \end{cases} \quad \sum_{i \in E} f(i)\pi_i = f(1)\pi_1 + f(2)\pi_2 + f(3)\pi_3 + \dots = \pi_3$$

- This is the reason MCMC works!

Markov chain Monte Carlo

- We are interested in some complicated distribution
- Artificially, we want to design a Markov chain with stationary distribution because then we can use ergodic theorem to approximate properties of this distribution e.g. mean, variance
 - This turns the objective function $E[h(\mathbf{x})] = \sum_{\mathbf{x} \in E} \pi_{\mathbf{x}} h(\mathbf{x})$ into $E[h(\mathbf{x})] \approx \frac{1}{N} \sum_{i=1}^N \pi_{\mathbf{x}} h(X_i)$
- Classical MC is very hard to implement in high dimensional spaces.
- MCMC can also be hard to implement, but we can normally formulate an MCMC algorithm

Metropolis -Hastings

We start with a target (posterior) distribution and given initial value $X_0 = x_0$, we construct a Markov chain using the following rules:

1. Start with initial value
2. **for** $n = 0 \rightarrow N$:
 1. Simulate candidate value $Y \sim \underbrace{q(j|X_n = i)}_{\substack{\text{proposal transition} \\ \text{density} / \text{jumping density}}} \quad \text{Suppose } Y = j$
 2. Compute the Metropolis Hastings **acceptance probability**

$$a_{ij} = \min \left\{ \frac{\overbrace{\pi_j q(i|j)}^{j \rightarrow i}}{\underbrace{\pi_i q(j|i)}_{i \rightarrow j}}, 1 \right\}$$

1. Remember that $\pi_j q(i|j)$ is the probability density of being in at value j , multiplied by the probability of proposing the step $j \rightarrow i$
2. We can derive this term using the detailed balance (see below)
3. Note that we get the Metropolis algorithm when we assume a symmetrical target density (e.g. $q(j|i) \sim \mathcal{N}(i, \sigma)$), i.e., $\min\{\frac{\pi_j}{\pi_i}, 1\}$ which is saying that is it more probable that we draw a value from the target distribution π of j than i (it's just the Bayesian posterior)!
3. Generate $U \sim \text{Unif}[0, 1]$
4. Accept the candidate $Y = j$ if $U < a_{ij}$, otherwise set $X_{n+1} = X_n$

$$X_{n+1} = \begin{cases} Y & \text{if } U \leq a_{ij} \\ X_n & \text{if } U > a_{ij} \end{cases}$$

1. Note that if $\pi_j q(i|j) > \pi_i q(j|i)$ i.e.
3. **End for**
4. Return property e.g. $E[h(\mathbf{x})] \approx \frac{1}{N} \sum_{i=1}^N h(X_i)$
 - The MH acceptance probability satisfies the detailed balance requirement $\pi_i p_{ij} = \pi_j p_{ji}$
 - Let $\mathbf{P} = \{p_{ij}\}$ (the transition probability matrix) then:

$$\begin{aligned} p_{ij} &= \Pr(X_{n+1} = j | X_n = i) \\ &= \Pr(X_1 = j | X_0 = i) \\ &= a_{ij} q(j|i) \end{aligned}$$

$\therefore$

$$f_i p_{ij} = f_j p_{ji} \quad \text{Detailed balance requirement}$$

$$\frac{\pi_i}{NC} q(j|i) a_{ij} = \frac{\pi_j}{NC} q(i|j) a_{ji} \quad \text{where NC = normalizing constant}$$

$$\frac{a_{ij}}{a_{ji}} = \frac{\pi_j q(i|j)}{\pi_i q(j|i)}$$

- What the first part is saying is that the probability of going from $i \rightarrow j$ in 1 step is equal to the probability of proposing the step ($q(j|i)$) multiplied by the probability of accepting the proposal (a_{ij})!
- For the second part, we know that probabilities are bounded between $[0, 1]$, so:

- * If the $\text{RHS} < 1$, we can set the term a_{ij} equal to RHS , and $a_{ji} = 1$
- * If $\text{RHS} \geq 1$, we can set the term $a_{ij} = 1$, and $a_{ji} = \frac{1}{\text{RHS}}$
- * We pick the minimum value, because if $\text{RHS} < 1$, then it means that the probability of accepting the new value $j|i$ is < 1 , and if $\text{RHS} \geq 1$, then we always accept the new value!
- * Note that we can't sample from the true posterior f_i , but we know our target density π_i is proportional to it via some normalizing constant, hence, in the ratio of the acceptance probabilities, they cancel out.
- * In the last line, we set $a_{ji} = 1$
- * Note that these equations may also be written in somewhat more intuitive notation that relates to Bayes' theorem (using the Metropolis sampler as a simple example) and the likelihood ratio (see Ben Lambert's video)

$$\begin{aligned}
P(\theta|x) &= \frac{P(x|\theta)P(\theta)}{P(x)} \\
&\propto P(x|\theta)P(\theta) \\
\frac{P(\theta'_t|x)}{P(\theta_{t-1}|x)} &= \frac{P(x|\theta'_t)P(\theta'_t)}{P(x|\theta_{t-1})P(\theta_{t-1})} \\
\text{where: } \theta'_t &\sim \mathcal{N}(\theta_{t-1}, \sigma) \therefore P(\theta'_t) = P(\theta_{t-1}) \\
\frac{P(\theta'_t|x)}{P(\theta_{t-1}|x)} &= \frac{P(x|\theta'_t)}{P(x|\theta_{t-1})}
\end{aligned}$$

- To make it into the continuous case, we replace proposal probabilities $q(j|i)$ with proposal densities $q(\mathbf{y}|\mathbf{x})$, AKA **kernels**

$$a(\mathbf{x}, \mathbf{y}) = \min \left\{ \frac{f(\mathbf{y})q(\mathbf{x}|\mathbf{y})}{f(\mathbf{x})q(\mathbf{y}|\mathbf{x})}, 1 \right\}$$

- To ensure the Markov chain is irreducible, we can require $q(\mathbf{y}|\mathbf{x}) > 0$ for all $\mathbf{x}, \mathbf{y} \in E$, or $q(\mathbf{y}|\mathbf{x}) > \epsilon$ if $|\mathbf{x} - \mathbf{y}| < \delta$
- A common example of a proposal scheme is a random walk $Y = X_n + \epsilon_n$, which are always symmetric by convention and have the following properties
 - It's independent of X_n
 - $E(\epsilon_n) = 0$
 - $q(y|x) = q(|y - x|)$